```
import numpy as np
from scipy import stats
import pandas as pd
```

Question #1

a. Generate a (5, 5) NumPy array of random numbers from normal distribution with mean = 50, std = 10

```
numbers_array = np.random.normal(50, 10, (5, 5))
print(numbers_array)

mean = 50
std = 10
```

```
[[36.3241601  41.77256436 65.24398795 50.83984436 54.28428795]
 [51.26477673 25.05666703 52.41826115 55.46279851 38.76965897]
 [33.98954446 50.69413989 42.88047292 46.79918415 66.36509688]
 [63.21505271 39.67066573 67.92998184 49.56111631 53.27080181]
 [54.469133   56.14663144 36.46908944 58.7679768  54.90914513]]
```

b. Compute: • Mean of each row • Standard deviation of each column • Global min and max • Median and mode of the entire array

```
mean_rows = np.mean(numbers_array, axis=1)
print("Mean of each row:",mean_rows)

std_cols = np.std(numbers_array, axis=0)
print("Standard deviation of each column:",std_cols)

global_min = np.min(numbers_array)
print("Global minimum:",global_min)

global_max = np.max(numbers_array)
print("Global maximum:",global_max)

median = np.median(numbers_array)
print("Median of the entire array:",median)

array_mode = stats.mode(numbers_array, axis=None)
print("Mode of the entire array:",array_mode) #50.0
```

```
Mean of each row: [49.69296894 44.59443248 48.14568766 54.72952368 52.15239516]
Standard deviation of each column: [11.10419423 10.63989937 12.23791791  4.28321321  8.77661705]
Global minimum: 25.05666703473047
Global maximum: 67.92998184092033
Median of the entire array: 51.26477673437334
Mode of the entire array: ModeResult(mode=np.float64(25.05666703473047), count=np.int64(1))
```

c. Normalize the array values using Z-score normalization: $z = (x - \mu) / \sigma$

```
z_normalized_array = (numbers_array - np.mean(numbers_array)) / np.std(numbers_array)

print(z_normalized_array)
```

```
[[-1.27197112 -0.76009476  1.44504022  0.09177416  0.41537886]
 [ 0.13169646 -2.33054956  0.24006606  0.52609966 -1.04221705]
 [-1.49130773  0.07808526 -0.656007   -0.28784496  1.55036815]
 [ 1.25442221 -0.95756766  1.69738874 -0.02836205  0.32016207]
 [ 0.43274501  0.59034561 -1.25835504  0.83662042  0.47408405]]
```

d. Apply a thresholding operation: • Set values < mean to 0, ≥ mean to 1 (create binary array) • Sort each row individually and print the sorted matrix

```
binary_array = np.where(numbers_array < mean, 0, 1)
print('Binary Array:\n', binary_array)

sorted_matrix = np.sort(numbers_array, axis=1)
print("\nSorted Matrix:\n",sorted_matrix)
```

```
Binary Array:
 [[0 0 1 1 1]
 [1 0 1 1 0]
 [0 1 0 0 1]
 [1 0 1 0 1]
 [1 1 0 1 1]]

Sorted Matrix:
 [[36.3241601  41.77256436 50.83984436 54.28428795 65.24398795]
 [25.05666703 38.76965897 51.26477673 52.41826115 55.46279851]
 [33.98954446 42.88047292 46.79918415 50.69413989 66.36509688]
 [39.67066573 49.56111631 53.27080181 63.21505271 67.92998184]
 [36.46908944 54.469133   54.90914513 56.14663144 58.7679768 ]]
```

Question #2

You're working on comparing two sets of sensor alert codes logged from different subsystems in a smart factory. a. Generate: • codes_A: 2D NumPy array of shape (6, 6) with random integers from 10 to 40 • codes_B: 2D NumPy array of shape (6, 6) with random integers from 25 to 55

```
codes_A = np.random.randint(10, 41, (6, 6))
print("Codes A:")
print(codes_A)

codes_B = np.random.randint(25, 56, (6, 6))
print("\nCodes B:")
print(codes_B)
```

```
Codes A:
[[27 21 37 10 22 36]
 [16 14 39 19 27 31]
 [29 38 36 35 18 10]
 [25 25 15 38 40 14]
 [37 29 11 22 20 38]
 [22 36 20 37 18 14]]

Codes B:
[[36 31 50 49 53 52]
 [39 52 44 47 54 50]
 [44 55 55 29 35 55]
 [32 51 53 54 47 33]
 [45 38 45 27 52 55]
 [53 30 46 52 55 26]]
```

b. Perform the following element-wise operations: • Minimum, Maximum • Modulo: codes_B % codes_A (handle division by zero errors if needed) • Boolean matrix: codes_A is even and codes_B is odd

```
max_matrix = np.maximum(codes_A, codes_B)
min_matrix = np.minimum(codes_A, codes_B)

print("Maximum Matrix:")
print(max_matrix)

print("\nMinimum Matrix:")
print(min_matrix)

with np.errstate(divide='ignore', invalid='ignore'):
    modulo_matrix = np.where(codes_A != 0, codes_B % codes_A, np.nan)
    print("\nModulo Matrix (Codes B % Codes A) with NaN for division by zero:")
    print(modulo_matrix)

boolean_matrix = (codes_A % 2 == 0) & (codes_B % 2 == 1)
print("\nBoolean Matrix (Codes A are even and Codes B are odd):")
print(boolean_matrix)
```

```
Maximum Matrix:
[[36 31 50 49 53 52]
 [39 52 44 47 54 50]
 [44 55 55 35 35 55]
 [32 51 53 54 47 33]
 [45 38 45 27 52 55]
 [53 36 46 52 55 26]]

Minimum Matrix:
```

```
[[27 21 37 10 22 36]
 [16 14 39 19 27 31]
 [29 38 36 29 18 10]
 [25 25 15 38 40 14]
 [37 29 11 22 20 38]
 [22 30 20 37 18 14]]

Modulo Matrix (Codes B % Codes A) with NaN for division by zero:
[[ 9. 10. 13.  9.  9. 16.]
 [ 7. 10.  5.  9.  0. 19.]
 [15. 17. 19. 29. 17.  5.]
 [ 7.  1.  8. 16.  7.  5.]
 [ 8.  9.  1.  5. 12. 17.]
 [ 9. 30.  6. 15.  1. 12.]]

Boolean Matrix (Codes A are even and Codes B are odd):
[[False False False  True  True False]
 [ True False False False False False]
 [False  True  True False  True  True]
 [False False False False  True  True]
 [False False False  True False  True]
 [ True False False False  True False]]
```

c. Flatten both arrays and perform advanced set operations: • Sorted difference: elements in codes_A not in codes_B • Sorted intersection • Sorted union • Symmetric difference: values in either array but not in both

```
flatten_codesA = np.sort(codes_A.flatten())
print("Codes A - Flattened:", flatten_codesA)
flatten_codesB = np.sort(codes_B.flatten())
print("\nCodes B - Flattened:", flatten_codesB)

sorted_difference = np.setdiff1d(flatten_codesA, flatten_codesB)
print("\nSorted Difference (Elements in Codes A not in Codes B):", sorted_difference)

sorted_intersection = np.intersect1d(flatten_codesA, flatten_codesB)
print("\nSorted Intersection:", sorted_intersection)

sorted_union = np.union1d(flatten_codesA, flatten_codesB)
print("\nSorted Union:", sorted_union)

symmetric_difference = np.setxor1d(flatten_codesA, flatten_codesB)
print("\nSymmetric Difference (Values in either array but not in both):", symmetric_difference)
```

```
Codes A - Flattened: [10 10 11 14 14 14 15 16 18 18 19 20 20 21 22 22 22 25 25 27 27 29 29 31
 35 36 36 36 37 37 37 38 38 38 39 40]

Codes B - Flattened: [26 27 29 30 31 32 33 35 36 38 39 44 44 45 45 46 47 47 49 50 50 51 52 52
 52 52 53 53 53 54 54 55 55 55 55 55]

Sorted Difference (Elements in Codes A not in Codes B): [10 11 14 15 16 18 19 20 21 22 25 37 40]

Sorted Intersection: [27 29 31 35 36 38 39]

Sorted Union: [10 11 14 15 16 18 19 20 21 22 25 26 27 29 30 31 32 33 35 36 37 38 39 40
 44 45 46 47 49 50 51 52 53 54 55]

Symmetric Difference (Values in either array but not in both): [10 11 14 15 16 18 19 20 21 22 25 26 30 32 33 37 40 44 45 46 47 49 50 51
 52 53 54 55]
```

d. Construct a logical mask where: • codes_A > 25 • codes_B < 45 • (codes_A + codes_B) % 3 == 0 • Use this mask to extract and print matching entries from codes_A.

```
mask = (codes_A > 25) & (codes_B < 45) & ((codes_A + codes_B) % 3 == 0)

matching_entries = codes_A[mask]
print("Matching Entries from Codes A:", matching_entries)
```

```
Matching Entries from Codes A: [27 36]
```

e. Create a (6×1) vector V = np.arange(6).reshape(6, 1) and use broadcasting to:
• Add it to each column of codes_A • Subtract it from each row of codes_B • Explain the resulting shape and how broadcasting worked

```
vector_v = np.arange(6).reshape(6,1)
print("Vector V:")
print(vector_v)
```

```
print("\nAdding Vector V to each column of Codes A:")
print(codes_A + vector_v)

print("\nSubtracting Vector V from each row of Codes B:")
print(codes_B - vector_v)

print("\nThe resulting shape is (6, 6) because broadcasting aligns the dimensions of the vector and codes.\nVector V had a (6,1) shape and th
print("For addition, Vector V is broadcasted across the columns of Codes A.")
print("For subtraction, Vector V is broadcasted across the rows of Codes B.")
```

```
Vector V:
[[0]
 [1]
 [2]
 [3]
 [4]
 [5]]

Adding Vector V to each column of Codes A:
[[27 21 37 10 22 36]
 [17 15 40 20 28 32]
 [31 40 38 37 20 12]
 [28 28 18 41 43 17]
 [41 33 15 26 24 42]
 [27 41 25 42 23 19]]

Subtracting Vector V from each row of Codes B:
[[36 31 50 49 53 52]
 [38 51 43 46 53 49]
 [42 53 53 27 33 53]
 [29 48 50 51 44 30]
 [41 34 41 23 48 51]
 [48 25 41 47 50 21]]

The resulting shape is (6, 6) because broadcasting aligns the dimensions of the vector and codes.
Vector V had a (6,1) shape and the final result used Codes A or B to align the dimensions to a (6,6) shape by automatically expanding Ve
For addition, Vector V is broadcasted across the columns of Codes A.
For subtraction, Vector V is broadcasted across the rows of Codes B.
```

Question #3

```
data = {
'Category': ['Electronics', 'Electronics', 'Electronics', 'Furniture', 'Furniture', 'Clothing', 'Clothing',
'Clothing'],
'Subcategory': ['Phones', 'Laptops', 'TVs', 'Chairs', 'Desks', 'Men', 'Women', 'Kids'],
'Year': [2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021],
'Sales_2021': [150000, 200000, 180000, 95000, 110000, 80000, 90000, 50000],
'Sales_2022': [160000, 210000, 175000, 98000, 120000, 85000, 95000, 55000],
'Sales_2023': [170000, 205000, 190000, 105000, 125000, 90000, 99000, 60000],
'Discount (%)': [5, 10, 7, 5, 6, 8, 7, 5],
'Profit_2023': [34000, 38000, 35500, 21000, 25000, 17000, 18500, 11000]
}
df_products = pd.DataFrame(data)
print(df_products)
```

```
       Category Subcategory  Year  Sales_2021  Sales_2022  Sales_2023  \
0  Electronics      Phones  2021      150000      160000      170000
1  Electronics     Laptops  2021      200000      210000      205000
2  Electronics         TVs  2021      180000      175000      190000
3    Furniture      Chairs  2021       95000       98000      105000
4    Furniture       Desks  2021      110000      120000      125000
5     Clothing         Men  2021       80000       85000       90000
6     Clothing       Women  2021       90000       95000       99000
7     Clothing        Kids  2021       50000       55000       60000

   Discount (%)  Profit_2023
0             5        34000
1            10        38000
2             7        35500
3             5        21000
4             6        25000
5             8        17000
6             7        18500
7             5        11000
```

a. Multi-Indexing:

- Set a multi-index using 'Category', 'Subcategory', and 'Year'.
- Then, unstack the 'Year' level to move years to columns.

```
index = df_products.set_index(['Category', 'Subcategory', 'Year'])
print(index)

unstacked = index.unstack('Year')
print("\nUnstacked Year:", unstacked)
```

```
                             Sales_2021  Sales_2022  Sales_2023  \
Category    Subcategory Year
Electronics Phones      2021      150000      160000      170000
            Laptops     2021      200000      210000      205000
            TVs         2021      180000      175000      190000
Furniture   Chairs      2021       95000       98000      105000
            Desks       2021      110000      120000      125000
Clothing    Men         2021       80000       85000       90000
            Women       2021       90000       95000       99000
            Kids        2021       50000       55000       60000

                             Discount (%)  Profit_2023
Category    Subcategory Year
Electronics Phones      2021             5        34000
            Laptops     2021            10        38000
            TVs         2021             7        35500
Furniture   Chairs      2021             5        21000
            Desks       2021             6        25000
Clothing    Men         2021             8        17000
            Women       2021             7        18500
            Kids        2021             5        11000

Unstacked Year:                         Sales_2021 Sales_2022 Sales_2023 Discount (%)  \
Year                          2021       2021       2021         2021
Category    Subcategory
Clothing    Kids             50000      55000      60000            5
            Men              80000      85000      90000            8
            Women            90000      95000      99000            7
Electronics Laptops         200000     210000     205000           10
            Phones          150000     160000     170000            5
            TVs             180000     175000     190000            7
Furniture   Chairs           95000      98000     105000            5
            Desks           110000     120000     125000            6

                          Profit_2023
Year                             2021
Category    Subcategory
Clothing    Kids                11000
            Men                 17000
            Women               18500
Electronics Laptops             38000
            Phones              34000
            TVs                 35500
Furniture   Chairs              21000
            Desks               25000
```

Sales Growth:

- Calculate the year-over-year percent growth from 2021 → 2022 and 2022 → 2023 for each product (subcategory).
- Display as new columns 'Growth_21_22 (%)' and 'Growth_22_23 (%)'.

```
df_products['Growth_21_22 (%)'] = (df_products['Sales_2022'] - df_products['Sales_2021']) / df_products['Sales_2021'] * 100
df_products['Growth_22_23 (%)'] = (df_products['Sales_2023'] - df_products['Sales_2022']) / df_products['Sales_2022'] * 100

print("DataFrame with Year-over-Year Growth:\n", df_products[['Subcategory', 'Growth_21_22 (%)', 'Growth_22_23 (%)']])
```

```
DataFrame with Year-over-Year Growth:
   Subcategory  Growth_21_22 (%)  Growth_22_23 (%)
0      Phones          6.666667          6.250000
1     Laptops          5.000000         -2.380952
2         TVs         -2.777778          8.571429
3      Chairs          3.157895          7.142857
4       Desks          9.090909          4.166667
5         Men          6.250000          5.882353
6       Women          5.555556          4.210526
7        Kids         10.000000          9.090909
```

Discount Impact: For each product, compute effective revenue in 2023 as:

- Effective Revenue = Sales_2023 × (1 − Discount (%) / 100)

```
df_products['Effective Revenue'] = df_products['Sales_2023'] * (1 - df_products['Discount (%)'] / 100)

print("Effective Revenue For Each Product:\n", df_products[['Category', 'Subcategory', 'Effective Revenue']])
```

```
Effective Revenue For Each Product:
       Category Subcategory  Effective Revenue
0  Electronics      Phones           161500.0
1  Electronics     Laptops           184500.0
2  Electronics         TVs           176700.0
3    Furniture      Chairs            99750.0
4    Furniture       Desks           117500.0
5     Clothing         Men            82800.0
6     Clothing       Women            92070.0
7     Clothing        Kids            57000.0
```

Profit Margin Analysis: a. Compute 2023 profit margin as a percentage: • Profit Margin = Profit_2023 / Effective Revenue × 100

```
df_products['Profit Margin (%)'] = (df_products['Profit_2023'] / df_products['Effective Revenue']) * 100

print("Profit Margin For Each Product:\n", df_products[['Category', 'Subcategory', 'Profit Margin (%)']])
```

```
Profit Margin For Each Product:
       Category Subcategory  Profit Margin (%)
0  Electronics      Phones          21.052632
1  Electronics     Laptops          20.596206
2  Electronics         TVs          20.090549
3    Furniture      Chairs          21.052632
4    Furniture       Desks          21.276596
5     Clothing         Men          20.531401
6     Clothing       Women          20.093407
7     Clothing        Kids          19.298246
```

Pivot Summary: a. Create a pivot table where rows are 'Category' and columns are 'Year', showing the total sales per category per year. b. Use .pivot_table() with aggregation function = sum

```
pivot_table = df_products.pivot_table(index='Category', values=['Sales_2021', 'Sales_2022', 'Sales_2023'], aggfunc='sum')

print("Pivot Table (Total Sales Per Category Per Year):\n\n", pivot_table)
```

```
Pivot Table (Total Sales Per Category Per Year):

                Sales_2021  Sales_2022  Sales_2023
Category
Clothing         220000      235000      249000
Electronics      530000      545000      565000
Furniture        205000      218000      230000
```

Insights: a. Find the subcategory with the highest growth from 2021 to 2023. b. Find the category with the lowest average profit margin in 2023

```
subcategory_highest_growth = df_products.loc[(df_products['Growth_21_22 (%)'] + df_products['Growth_22_23 (%)']).idxmax(), 'Subcategory']
print("Subcategory with the Highest Growth from 2021 to 2023:", subcategory_highest_growth)

category_lowest_profit_margin = df_products.groupby('Category')['Profit Margin (%)'].mean().idxmin()
print("Category with the Lowest Average Profit Margin in 2023:", category_lowest_profit_margin)
```

```
Subcategory with the Highest Growth from 2021 to 2023: Kids
Category with the Lowest Average Profit Margin in 2023: Clothing
```

Question #4

```
A1 = np.array([
[5, 8, 2],
[6, 7, 3],
[4, 2, 9]
])

A2 = np.array([
[2, 6, 1],
[9, 5, 4],
```

```
[3, 7, 8]
])

A3 = np.array([
[4, 3],
[7, 5],
[6, 2]
])

print('A1:')
print(A1)
print('\nA2:')
print(A2)
print('\nA3:')
print(A3)
```

```
A1:
[[5 8 2]
 [6 7 3]
 [4 2 9]]

A2:
[[2 6 1]
 [9 5 4]
 [3 7 8]]

A3:
[[4 3]
 [7 5]
 [6 2]]
```

a) Find the diagonal elements of A1 and A2 as 1D arrays

```
diagonal_A1 = np.diag(A1)
diagonal_A2 = np.diag(A2)

print("Diagonal elements of A1:", diagonal_A1)
print("Diagonal elements of A2:", diagonal_A2)
```

```
Diagonal elements of A1: [5 7 9]
Diagonal elements of A2: [2 5 8]
```

b) Compute the matrix multiplication of A2 and A3.

```
A2_A3_product = np.dot(A2, A3)
print("Matrix multiplication of A2 and A3:\n", A2_A3_product)
```

```
Matrix multiplication of A2 and A3:
 [[ 56  38]
 [ 95  60]
 [109  60]]
```

c) Compute the sum of diagonal elements of A1 and A2.

```
sum_diagonal_A1 = np.sum(diagonal_A1)
sum_diagonal_A2 = np.sum(diagonal_A2)

sum_diagonal_A1_A2 = sum_diagonal_A1 + sum_diagonal_A2

print("Sum of diagonal elements of A1:", sum_diagonal_A1_A2)
```

```
Sum of diagonal elements of A1: 36
```

d) Compute the determinants of A1, A2, and ($A3^t \times A3$).

```
determinant_A1 = np.linalg.det(A1)
determinant_A2 = np.linalg.det(A2)
determinant_A3_transpose_A3 = np.linalg.det(np.dot(A3.T, A3))

print("Determinant of A1:", determinant_A1)
print("Determinant of A2:", determinant_A2)
print("Determinant of (A3^t × A3):", determinant_A3_transpose_A3)
```

```
Determinant of A1: -83.00000000000003
Determinant of A2: -288.00000000000006
Determinant of (A3ᵗ × A3): 357.00000000000017
```

e) Compute the eigenvalues and eigenvectors of A1 and A2

```
eigenvalues_A1, eigenvectors_A1 = np.linalg.eig(A1)
eigenvalues_A2, eigenvectors_A2 = np.linalg.eig(A2)

print("Eigenvalues of A1:", eigenvalues_A1)
print("Eigenvectors of A1:\n", eigenvectors_A1)
print("\nEigenvalues of A2:", eigenvalues_A2)
print("Eigenvectors of A2:\n", eigenvectors_A2)
```

```
Eigenvalues of A1: [15.38414104 -0.83619378  6.45205274]
Eigenvectors of A1:
 [[-0.57368743 -0.81334885 -0.40660143]
 [-0.60723684  0.53801714 -0.29036006]
 [-0.54967824  0.22136216  0.86623687]]

Eigenvalues of A2: [14.98540589 -4.37662264  4.39121675]
Eigenvectors of A2:
 [[ 0.3301887   0.64137299 -0.37137611]
 [ 0.5920846  -0.72396526 -0.29474944]
 [ 0.73512669  0.25399804  0.88045588]]
```

f) Compute the inverse of A1 and A2 (if possible).

```
inverse_A1 = np.linalg.inv(A1) if np.linalg.det(A1) != 0 else "Not invertible"

inverse_A2 = np.linalg.inv(A2) if np.linalg.det(A1) != 0 else "Not invertible"

print("Inverse of A1:\n", inverse_A1)
print("\nInverse of A2:\n", inverse_A2)
```

```
Inverse of A1:
 [[-0.68674699  0.81927711 -0.12048193]
 [ 0.5060241  -0.44578313  0.03614458]
 [ 0.19277108 -0.26506024  0.15662651]]

Inverse of A2:
 [[-0.04166667  0.14236111 -0.06597222]
 [ 0.20833333 -0.04513889 -0.00347222]
 [-0.16666667 -0.01388889  0.15277778]]
```

g) Compute the Moore-Penrose pseudoinverse of A1 and A2.

```
pseudoinverse_A1 = np.linalg.pinv(A1)
pseudoinverse_A2 = np.linalg.pinv(A2)

print("Moore-Penrose pseudoinverse of A1:\n", pseudoinverse_A1)
print("\nMoore-Penrose pseudoinverse of A2:\n", pseudoinverse_A2)
```

```
Moore-Penrose pseudoinverse of A1:
 [[-0.68674699  0.81927711 -0.12048193]
 [ 0.5060241  -0.44578313  0.03614458]
 [ 0.19277108 -0.26506024  0.15662651]]

Moore-Penrose pseudoinverse of A2:
 [[-0.04166667  0.14236111 -0.06597222]
 [ 0.20833333 -0.04513889 -0.00347222]
 [-0.16666667 -0.01388889  0.15277778]]
```

h) Compute the QR decomposition of A1 and A2.

```
qr_A1 = np.linalg.qr(A1)
qr_A2 = np.linalg.qr(A2)

print("QR decomposition of A1:\n", qr_A1)
print("\nQR decomposition of A2:\n", qr_A2)
```

```
QR decomposition of A1:
 QRResult(Q=array([[-0.56980288,  0.6274524 ,  0.5306865 ],
       [-0.68376346, -0.00377983, -0.72969394],
```

```
       [-0.45584231, -0.77864575,  0.43118278]]), R=array([[ -8.77496439, -10.25645188,  -7.29347689],
       [  0.        ,   3.43586886,  -5.76424643],
       [  0.        ,   0.        ,   2.75293623]]))

QR decomposition of A2:
 QRResult(Q=array([[-0.20628425,  0.64505304, -0.73576721],
       [-0.92827912, -0.36679487, -0.06131393],
       [-0.30942637,  0.67034924,  0.67445327]]), R=array([[-9.69535971, -8.04508572, -6.39481173],
       [ 0.        ,  6.72878858,  4.54066748],
       [ 0.        ,  0.        ,  4.41460324]]))
```

i) Compute the Singular Value Decomposition (SVD) of A1 and A2

```
singular_value_decomposition_A1 = np.linalg.svd(A1)
singular_value_decomposition_A2 = np.linalg.svd(A2)

print("Singular Value Decomposition of A1:\n", singular_value_decomposition_A1)
print("\nSingular Value Decomposition of A2:\n", singular_value_decomposition_A2)
```

```
Singular Value Decomposition of A1:
 SVDResult(U=array([[-0.58794308, -0.46181379, -0.66411667],
       [-0.61498459, -0.27813709,  0.73785752],
       [-0.52546826,  0.84223974, -0.12047958]]), S=array([15.41081936,  7.06580781,  0.76223791]), Vh=array([[-0.5665822 , -0.65274747,
       [-0.08618017, -0.56001955,  0.82398488],
       [ 0.81948617, -0.51019492, -0.26104322]]))

Singular Value Decomposition of A2:
 SVDResult(U=array([[-0.35118053,  0.15065183, -0.92410836],
       [-0.65881962, -0.74106164,  0.12955443],
       [-0.66530365,  0.65431771,  0.3594988 ]]), S=array([15.64108305,  5.33215333,  3.45321037]), Vh=array([[-0.55160174, -0.64306971,
       [-0.82617616,  0.33360383,  0.45402801],
       [ 0.11475396, -0.68932563,  0.71530532]]))
```